目录

0. 一页速览（TL;DR） 3

1. 体系总览：谁来决策、谁来执行 3

1.1 数据流与调用链 . . . 3

1.2 职责边界表（谁管什么） . . . 4

1.3 概念词典（读后文必备） . . . 4

2. 核心接口 `TrendStrategy` 5

2.1 接口源码（`strategy/types.ts` 摘录，签名逐字） . . . 5

2.2 方法逐条契约（何时被调、你要做什么、什么别做） . . . 6

2.3 tick 主循环调用时序（策略视角） . . . 7

3. 数据契约：策略能看到什么、要回答什么 7

3.1 `UnifiedTrendSignal`（`trend-watch/signal.ts`，策略输入的主数据） . . . 7

3.2 `VolatilityFeatures`（`trend-watch/indicators.ts`） . . . 8

3.3 `HtfAssessment`（高周期背景，`htf5m` / `htf15m`） . . . 9

3.4 `StrategyMarketCtx` 与 `StrategyPositionCtx` . . . 9

3.5 `TrendTradeMetaRow` 里策略离场评估可复用的字段 . . . 10

3.6 输出规范：modeTag 与 reason 文案 . . . 10

4. 基础设施硬规则：策略的自由度边界 10

4.1 开仓守卫链（`manageEntries`：在问你 evaluateEntry 之前，基础设施已经做过的筛选） . . . 10

4.2 二次确认（待入场）与直开路径 . . . 11

4.3 真实开仓执行链（`executeOpen`，pending 转换与直开共用） . . . 11

4.4 退出优先级（`manageExits`，从上到下短路） . . . 11

4.5 轮次体系速查 . . . 12

4.6 策略能做 / 不能做（红线清单的展开版） . . . 12

5. 实战示例：手写一个新算法（pullback-v1 · 趋势回撤跟随） 13

5.1 算法思路（一段话） . . . 13

5.2 完整代码（`strategy/pullback.ts`，可直接编译） . . . 13

5.3 逐段讲解（为什么这么写） . . . 18

6. 接入上线四步（含 bump 纪律） 19

6.1 第一步 · 实现算法 . . . 19

6.2 第二步 · 注册（唯一入口） 19
6.3 第三步（可选）· 参数接进控制台 19
6.4 第四步 · bump 引擎单例（改行为必 bump，铁律） 20
6.5 验证与回滚（不重启 server） 20
7. 运行期管理：切换、调参与审计 **21**
7.1 切换算法（UI / control API） 21
7.2 参数控制台 21
7.3 审计三查（排查「为什么没开/为什么开了/为什么平了」） 21
7.4 FAQ 22
8. 红线清单与上线自测 **22**
8.1 红线（逐条自查） 22
8.2 上线前自测清单 23
8.3 版本记录 23

> **文档版本** v1（2026-10-17） · 适用代码：`src/lib/market/trend-trade/`（enhanced 引擎单例 v15 / paper v9 / paperAuto v2 时代） **读者**：要往「趋势自动模拟交易」里加新算法的人（主要是我们自己） **一句话**：新算法 = 实现一个 `TrendStrategy` 接口 + 在注册表登记一行 + bump 引擎单例 key，UI 下拉自动出现，资金/风控/幂等/审计全部不用管。

0. 一页速览（TL;DR）

你要做的事	在哪里	工作量
实现算法逻辑	新文件 `src/lib/market/trend-trade/strategy/<你的算法>.ts`	主要工作量
注册进工厂表	`strategy/registry.ts` 的 `STRATEGY_FACTORIES` 加一行	1 行
（可选）参数进控制台	`console-params.ts` 的 `PARAM_DEFS` 加条目	每参数一条
上线（不重启 server！）	`enhanced/engine.ts` 单例 key bump v15→v16（三处代码 + 头部注释）	4 处小改
验证	热重载后 `GET /api/trend-trading` 看 `strategies` 列表 + UI 下拉切换	2 分钟

参考实现：`strategy/short-swing.ts`（短周期套利 v1，生产在用）——新算法照它的骨架写即可。本文 §5 有一份带逐段讲解的完整教学示例。

三句话理解体系：

1. **选币不归算法管**。哪些币值得看，由异动候选池（TrendWatchService）决定；算法拿到的是池内品种的「统一趋势信号」。
2. **资金与风控不归算法管**。保证金怎么算、最多几仓、重复开仓怎么防、强平怎么巡检、审计怎么落库，全部由 AutoPaperTrader（基础设施）统一提供。
3. **算法只回答两个问题**：这个币**现在想不想开仓、往哪个方向**（`evaluateEntry`）；这笔持仓**现在要不要离场**（`evaluateExit`）。答案不同，策略就不同——这就是全部个性所在。

1. 体系总览：谁来决策、谁来执行

1.1 数据流与调用链

```
币安/Bybit/Gate 多源行情（futures-collector，全局单例，后端天然共享）
        │ 1m K 线 + 标记价 + 异动事件
        ▼
TrendWatchService（异动候选池 + 1m/5m/15m 趋势状态机）
        │ 统一趋势信号 UnifiedTrendSignal（唯一趋势来源，不重复计算）
        ▼
AutoPaperTrader（基础设施，SCAN_MS = 10s 主循环）
        │ ①强平巡检 ②MFE/MAE 采集 ③退出管理 ③b observe 喂价 ④权益峰值
        │ ⑤开仓管理（待入场复核 → 新信号扫描）
```

```
        ▼
TrendStrategy（你的算法，可插拔）
        │ evaluateEntry / evaluateExit / nextRound / onOpen / onClose
        ▼
PaperEngine（模拟撮合，账户 trend-auto-v1，20x 杠杆、费率 0.02% 双边）
        │
        ▼
SQLite: PaperAccount/PaperPosition/PaperTrade + TrendTradeMeta /
        TrendPendingEntry / TrendAutoDecision / TrendStrategyState
```

要点：**策略实例住在 AutoPaperTrader 里**，而 AutoPaperTrader 住在 enhanced 引擎单例里（`enhanced/engine.ts`，当前 `__binanceRadarEnhanced_v15`）。所以改任何策略代码都属于「改引擎行为」，上线必须 bump 单例 key（§6.4）——否则热重载后 `globalThis` 上挂的还是旧代码构造的旧实例，你的新逻辑一行都不会生效。

1.2 职责边界表（谁管什么）

事项	归属	说明
选币（哪些币进入视野）	基础设施	异动候选池供池，策略不自己扫全市场
趋势方向/相位/评分	基础设施	TrendWatchService 唯一来源，策略**只读不另算**
开仓方向、时机、模式	**策略**	`evaluateEntry` 全权判定
离场（策略性）	**策略**	`evaluateExit`：冲高落袋/走弱先走等
离场（模型性）	基础设施	ENDED 周期匹配、反向确认、强平——永远先于策略执行
保证金/杠杆/仓位上限	基础设施	权益 × min(比例, 95%÷持仓数)，固定 20x，控制台可调
二次确认/冷却/容量/幂等	基础设施	待入场复核 + `{cycleId}#R{round}` 决策键防重
手续费/滑点	基础设施	双边各 0.02%（Task 59 定版）+ 2bp 滑点估算
审计与持久化	基础设施 + 策略配合	决策日志/跳过记录基础设施写；轮次状态策略经 `bindPersistence` 落库
接真实下单	**谁都不许**	纯模拟红线；只作用于独立账户 trend-auto-v1

1.3 概念词典（读后文必备）

- **tick / 主循环**：AutoPaperTrader 每 10 秒（`SCAN_MS`）跑一轮的固定流程（§2.3 有全序图）。
- **趋势周期（cycle）**：`{symbol}:{L|S}:{确认时刻ms}` 标识的「一次趋势 episode」。同 symbol 同方向从确认到结束算一个周期，是持仓关联与幂等的锚。
- **轮次（round）**：同一周期内「开仓→平仓→再开仓」的第几趟，决策键 = `{cycleId}#R{round}`。走弱先走、走强再进就是我们定的轮次玩法，上限由策略约束。
- **待入场（pending）**：策略第一次说「想开」后并不马上开，先记一条待入场记录，等**下一根已收盘 K 线**出现后全量复核（含重新询问策略），通过才真开。控制台可关（`waitBarConfirm`），关了就是当根直开。

- **清场**：切换算法时先把当前算法的全部持仓按标记价平掉、待入场作废——不同算法持仓管理规则不同，不混管。
- **bump**：改了交易链路行为后，把 enhanced 引擎单例 key 从 v15 升到 v16（§6.4），让热重载后的进程用新代码重建实例。**server 3000 永不重启**，这是我们的铁律。

2. 核心接口 TrendStrategy

2.1 接口源码（strategy/types.ts 摘录，签名逐字）

```
export interface TrendStrategy {
  /** 唯一 id（持久化到 TrendAutoConfig.strategyId；建议 {算法名}-v{n}） */
  readonly id: string;
  /** 展示名（UI 下拉显示） */
  readonly name: string;
  /** 一句话说明（UI 悬浮提示；切换确认弹窗也会展示） */
  readonly description: string;

  /**
   * 行情采样：每 tick 对池内/持仓 symbol 调用（mark>0 才会调用）。
   * 需要短窗价格序列的策略在此自维护样本；实现不得阻塞。
   */
  observe(symbol: string, mark: number, now: number): void;

  /** 入场评估（该 symbol 无持仓、信号 dataFresh 时调用） */
  evaluateEntry(signal: UnifiedTrendSignal, ctx: StrategyMarketCtx): StrategyEntryVerdict;

  /** 离场评估（该 symbol 有 OPEN 持仓时调用；基础设施离场规则在此之前已处理） */
  evaluateExit(input: {
    meta: TrendTradeMetaRow;
    pos: StrategyPositionCtx;
    mark: number;
    signal: UnifiedTrendSignal | null;
    now: number;
  }): StrategyExitVerdict;

  /** 本周期下一轮轮次号（1 = 首次开仓；同周期平仓后再开 = 2…） */
  nextRound(symbol: string, cycleId: string): number;

  /** 开仓成功回调（维护轮次状态；barTime = 开仓依据的已收盘 K 线） */
  onOpen(symbol: string, cycleId: string, barTime: number | null, now: number): void;

  /** 平仓成功回调（保留轮次状态供再入场判定；记录离场时刻/原因） */
  onClose(symbol: string, cycleId: string, reason: string, now: number): void;

  /** 导出全部 symbol 状态（持久化到 TrendStrategyState 表；重启恢复轮次语义） */
  exportState(): Record;
  /** 导入持久化状态（boot / 切换算法 / 控制台调参热装配时调用） */
  importState(raw: unknown): void;

  /** 绑定状态持久化回调（AutoPaperTrader 装配时调用一次；状态变化点调用它落库） */
  bindPersistence(fn: (strategyId: string, symbol: string, json: string) => void): void;
}
```

两个裁决对象的形状（返回什么、每个字段什么含义）：

```
/** 入场评估结论 */
export interface StrategyEntryVerdict {
  want: boolean; // true = 希望开仓（随后走「待入场 → 新收盘 K 线二次复核」流程）
  side: 'LONG' | 'SHORT' | null; // 方向（want=true 时必有）
  modeTag: string; // 入场模式标签（审计/展示；建议大写下划线，如 PULLBACK / TREND_ALIGNED）
  reason: string; // 拒绝原因或入场理由（中文，进审计日志——写人能看懂的话）
}

/** 离场评估结论 */
export interface StrategyExitVerdict {
  exit: boolean;
  reason: TrendExitReason | null; // 策略离场专用: 'SPIKE_TAKE' | 'WEAKNESS_EARLY'（见 §4.4）
  detail: string; // 中文理由（审计日志）
}
```

2.2 方法逐条契约（何时被调、你要做什么、什么别做）

成员	调用时机	你要做什么	红线
id/name/description	注册表与 UI 读取	起唯一 id（xxx-v1）、中文名、一句话说明	id 一经上线不要改（DB 按 id 持久化配置与状态）
observe	每 tick，对池内 + 持仓 symbol（mark>0）	维护你需要的短窗价格序列（样本 Map 自管）	必须瞬时返回，禁止 I/O；同粒度重复推送建议覆盖末位防堆积
evaluateEntry	仅当：相位 = CONFIRMED_UP/DOWN、dataFresh=true、该币无 OPEN 持仓、无 ACTIVE 待入场、容量/冷却守卫已通过	给出想不想开、方向、模式标签、人话理由	只用信号与 ctx 里「当时可用」的数据；禁止未来函数；**绝不抛错**
evaluateExit	持有 OPEN 持仓、mark>0、且基础设施离场规则（ENDED/反向）未触发	给出要不要走、离场标签、人话理由	同上；signal 类型可空但实现上恒非空，写防御即可
nextRound	开仓扫描前（组装幂等键）	返回该 symbol 在该周期的下一个轮次号（无状态则 1）	与 onOpen 维护的轮次状态保持自洽
onOpen	开仓成功落库后	轮次 +1 / 新周期初始化；记 barTime；触发持久化	不要在这里做交易逻辑，只记账
onClose	平仓成功（含强平/人工）后	记录离场时刻/原因；清不需要的样本	同周期才更新轮次（旧周期残响不动）；同样只记账
exportState/importState	boot / 切换算法 / 调参热装配	把 per-symbol 状态序列化/恢复	逐字段校验类型，脏数据丢弃（学 short-swing 的写法）；向后兼容自管
bindPersistence	AutoPaperTrader 装配时调用一次	把回调存起来，在 onOpen/onClose 等状态变化点调用	持久化失败要吞掉（不影响决策），不能反向抛错

2.3 tick 主循环调用时序（策略视角）

```
每 tick（10s），AutoPaperTrader 按固定顺序执行：
① 强平巡检 + 新强平终态补写 —— 策略不参与（任何模式都跑）
② 持仓 MFE/MAE 增量采集（写 meta） —— 策略不参与；但 evaluateExit 可读 meta.mfe 复用
②b 策略自愈：DB strategyId ≠ 实例 → 重新装配（恢复参数 + importState）
③ 退出管理 manageExits（每笔 OPEN 持仓）：
    ① 信号 phase=ENDED 且周期匹配且证据 K 线晚于开仓 K 线 → 平（TREND_ENDED）
    ② phase=END_PENDING 同周期 → 只记首次报警时刻（要不要提前走 → 轮到策略）
    ③ 策略 evaluateExit → exit=true 且 reason 非空 → 平（策略 reason）
    ④ 反向 CONFIRMED 且 dataFresh → 平（REVERSE_CONFIRMED）
③b 策略 observe：池内 signals + OPEN 持仓 symbol 的标记价逐一喂给策略
④ 权益峰值更新（回撤基准）
⑤ 仅 RUNNING 模式：
    ⑤a managePendings：待入场复核——等到了新收盘 K 线 → 全量重问 evaluateEntry
    ⑤b manageEntries：新信号扫描——CONFIRMED + dataFresh → 问 evaluateEntry
```

注意两个「策略不可见但影响语义」的细节：observe 在退出管理**之后**执行（冲高检测用的是截至上一轮的样本，粒度 ≈10s）；非确认相位在进策略**之前**就被基础设施静默跳过（不会刷你的拒绝审计）。

3. 数据契约：策略能看到什么、要回答什么

3.1 UnifiedTrendSignal（trend-watch/signal.ts，策略输入的主数据）

字段	类型	含义 / 策略怎么用
symbol	string	品种名（如 BTCUSDT）
phase	TrendPhaseV2	相位，见下表；**只有 CONFIRMED_* 会进你的 evaluateEntry**
direction	'up' | 'down' | 'flat'	确认趋势方向（CONFIRMED 系与 END_PENDING 相位下有意义）
trendCycleId	string | null	趋势周期标识（同 symbol 同方向一次 episode 唯一）；轮次/幂等的锚
trendStartedAt	number | null	当前周期确认时刻
endPendingSince	number | null	END_PENDING 起始时刻
endEvidenceBars	number	已累计的结束证据根数（不同已收盘 K 线）
endedAt / endReason	— / TrendEndReason	终态信息（CONFIRMED_REVERSAL / DECAY / STRUCTURE_INVALIDATION / DATA_EXIT）
score	number	UTE 趋势评分 0-100（启发式证据分；趋势起点分数不高，别拿 60+ 当入场线）
strength	number	方向强度 0-1（UTE returnStrength 口径）
confirmCount	number	UTE 连续确认计数（不同已收盘 K 线）
features	VolatilityFeatures	波动率与结构特征，见 §3.2

字段	类型	含义 / 策略怎么用
`htf5m` / `htf15m`	HtfAssessment	高周期背景评估，见 §3.3（`valid=false` 表示数据不足，**不得冒充有效背书**）
`htfAgree`	boolean | null	高周期方向与 1m 确认方向是否一致（仅 valid 时有意义）
`lastBarTime`	number | null	最近已参与评估的**已收盘** 1m K 线开盘时间（「新证据再入场」的判定基准）
`lastKlineAt`	number	最近 K 线拉取时刻
`dataFresh`	boolean	数据是否足以支持交易（行情有效 + K 线足够 + 未过期）；false 时基础设施已拦开仓
`dataUnavailableReason`	string | null	dataFresh=false 的原因（展示用）
`anomaly`	AnomalyContextDTO	异动上下文（dir/level/priority/active/lastAt）——**入池依据留痕，不参与方向判定**
`conflict`	boolean	异动方向与趋势方向冲突（明确展示，不静默覆盖）
`reason` / `changedAt`	string / number	信号的人话理由 / 信号生成时间

相位表（TrendPhaseV2）与策略机会：

phase	含义	策略能做什么
OBSERVING	观察中（UTE idle/未确认）	无（连进场评估都不会被问）
CANDIDATE_UP / CANDIDATE_DOWN	候选趋势（确认计数积累中）	无（方向未定，不开仓）
CONFIRMED_UP / CONFIRMED_DOWN	**已确认上涨/下跌趋势**	**唯一可开仓相位**；evaluateEntry 在此被问
END_PENDING	出现结束迹象、等待连续收盘 K 线确认（底层方向仍成立）	持仓者可评估「走弱先走」；不新开仓
ENDED	趋势结束（终态；基础设施按周期匹配强平离场）	无（基础设施已离场）
DATA_UNAVAILABLE	数据不足/行情过期：暂停判断，不伪装趋势结束	无（基础设施拦开仓、不误平仓）

3.2 `VolatilityFeatures`（`trend-watch/indicators.ts`）

字段	含义
`atr` / `atrPct`	ATR(14) 绝对值 / 占价比（%，跨币种可比的波动幅度）
`realizedVolPct`	已实现波动率（近期 1m 对数收益标准差，日化 % 表达）
`volPercentile`	上述 RV 在自身近期历史中的分位（0-1；样本不足 -1）
`volRegime`	波动状态分类

字段	含义
efficiencyRatio	**方向效率 ER =
upperWickPct / lowerWickPct	最近一根上/下影线占比（0-1）
breakout	'up' | 'down' | 'none'：最近收盘价相对前 lookbackBars-1 根高/低点的突破标记
falseBreakout	假突破：最近若干根内曾越出区间但收盘收回
barsUsed	参与计算的有效 bar 数（数据充分性）

3.3 HtfAssessment（高周期背景，htf5m / htf15m）

字段	含义
dir	'up' | 'down' | 'flat'（无状态评估，不维护高周期状态机）
efficiencyRatio / returnPct	高周期 ER 与区间收益
valid	已收盘高周期 bar 数是否达到最小确认要求（HTF_MIN_BARS=16）；**false 时不得当有效过滤用**
bars	已收盘高周期 bar 数

3.4 StrategyMarketCtx 与 StrategyPositionCtx

```
/** 每 tick 提供给策略的市场/账户上下文（全部为当前时刻快照，无未来数据） */
export interface StrategyMarketCtx {
  now: number;
  mark: number; // 当前标记价（多源供价快照；>0 保证可用）
  cfg: TrendAutoConfigRow; // 账户配置（模式/杠杆/持仓上限/marginRatio 等运行时真值）
  equity: { balance: number; marginUsed: number; unrealized: number;
            equity: number; openCount: number }; // 账户权益快照
  openSymbols: Set; // 当前有 OPEN 持仓的 symbol 集合
  lastCloseAt: number | null; // 该 symbol 最近一次平仓时刻（null = 从未平仓）
}

/** 持仓上下文（evaluateExit 用；来自 PaperPosition 当前快照） */
export interface StrategyPositionCtx {
  entryPrice: number;
  qty: number;
  margin: number;
  side: 'LONG' | 'SHORT';
  openedAt: number; // 开仓时刻（ms）
}
```

cfg（TrendAutoConfigRow）常用列：mode（RUNNING/PAUSED/STOPPED）、leverage（20）、maxPositions、marginRatio、slippageBps、strategyId、strategyParamsJson、waitBarConfirm、cooldownMs。策略可读但**不可写**，也不应绕过它自建配置。

3.5 TrendTradeMetaRow 里策略离场评估可复用的字段

基础设施每 tick 增量维护（无未来数据），策略可直接消费：

字段	含义	典型用法
meta.mfe / meta.mae	持仓期间最大有利/不利浮盈（占初始保证金 %，增量更新）	「浮盈从峰值回撤 N 个百分点 → 落袋」动态止盈
meta.mfeAt / meta.mfeDrawdown	MFE 出现时刻 / 峰值后当前回撤幅度	回撤速度/回撤深度的组合判断
meta.firstEndSignAt	本周期首次 END_PENDING 报警时刻	「报警超过 X 分钟还没恢复 → 先走」
meta.entryBarTime	开仓依据的已收盘 K 线	与 signal.lastBarTime 比较判断「有没有新证据」
meta.entryRound	本笔是第几轮（1 = 首次）	日志/统计按轮次分桶
meta.trendCycleId / meta.side / meta.entryEr 等	持仓关联周期/方向/开仓时 ER	周期匹配守卫（学 short-swing 的 END_PENDING 用法）

3.6 输出规范：modeTag 与 reason 文案

- **modeTag**：大写下划线、稳定不翻译，是统计分桶键。short-swing 现用：TREND_ALIGNED（5m 同向波段）/ RANGE_DIRECTION（5m 震荡选向）/ HTF_OPPOSED（拒绝标签）/ -（无模式）。新算法起自己的标签（如 PULLBACK），**同一算法内语义稳定**。
- **reason**：中文、一句话、写「为什么」。它是审计日志和「为什么没开仓」排查的第一现场（§7.3），拒绝和放行都要写清楚数字依据（如 评分 52 < 55（趋势起点门槛））。
- **离场 reason**：策略离场目前只能从基础设施预留的两个标签里选：SPIKE_TAKE（冲高落袋类）、WEAKNESS_EARLY（走弱先走类）。要加新标签需改 config.ts 的 TrendExitReason（改基础设施 = bump，见 §6.4）。

---

4. 基础设施硬规则：策略的自由度边界

> 原则：**基础设施保留的硬性安全规则优先于策略**。策略在这些规则之内表达个性，规则之外的空间一律没有。

4.1 开仓守卫链（manageEntries：在问你 evaluateEntry 之前，基础设施已经做过的筛选）

按序执行，任何一步不过都轮不到策略：

1. 相位守卫：phase ∈ {CONFIRMED_UP, CONFIRMED_DOWN}（否则静默跳过）；
2. 数据守卫：dataFresh = true（否则记跳过「数据不可用/过期」）；
3. 周期守卫：trendCycleId 必须存在；

4. 同币防御：该 symbol 已有 OPEN 持仓 → 跳过（同一币同时只有一笔）；

5. 待入场去重：该 symbol 已有 ACTIVE 待入场 → 跳过；

6. 轮次键组装：`round = strategy.nextRound(symbol, cycleId)` → `rKey = {cycleId}#R{round}`；该轮次 pending 已存在（ACTIVE/CONVERTED）→ 跳过（CANCELED 可删除重建）；

7. 容量守卫：`openCount + 待入场数 ≥ maxPositions` → 记跳过（不烧轮次）；

8. 冷却守卫：该 symbol 距上次平仓 < `cooldownMs`（控制台可调，默认 5 分钟）→ 记跳过；

9. 「先平后开」独立性：确认评估依据的收盘 K 线必须晚于该币上一笔平仓（同一根 K 线不既触发平仓又触发反向开仓）；

10. 标记价可用：`mark > 0`。

然后才调用 `evaluateEntry(signal, ctx)`。返回 `want=false` → 记跳过「策略不入场：{reason}」；`want=true` → 进入二次确认流程（§ 4.2）。

4.2 二次确认（待入场）与直开路径

- **waitBarConfirm = 开（默认）**：创建待入场记录（主键 `{cycleId}#R{round}`，快照 signal + modeTag + reason 落库），**不当根开仓**。之后每 tick 复核：等到「更新的已收盘 K 线」→ 全量重查（趋势未失效/数据新鲜/冷却/容量/资金）+ **重新询问策略 evaluateEntry**；通过 → 开仓；任何一项不满足 → 取消并记录原因；等待超过 `PENDING_EXPIRE_MS`（10 分钟）→ 超时取消。
- **waitBarConfirm = 关（控制台可调）**：当根直接走 `executeOpen`（容量/同币/资金/幂等全量守卫照跑），快 1 根 K 线，少一层防护。
- 设计意图：确认相位的形成只需要历史 K 线，但「入场那一刻」要用下一根 K 线再证一次，滤掉单根噪声/插针。

4.3 真实开仓执行链（`executeOpen`，pending 转换与直开共用）

> 容量复核 → 同币防御（再查一遍）→ 资金测算 → 幂等抢占 → `engine.open` → meta/审计 → 策略回调

资金测算口径（策略不需要做，但要知道约束）：费率读引擎实例 `feeRate`（自动账户 0.02%）；目标保证金 = 开仓时权益 × min(marginRatio, 95% ÷ maxPositions)；保证金 + 开仓手续费 > 可用余额 → 跳过并落库审计（**不变相提高杠杆或比例**）；保证金 < 2 USDT（MIN_MARGIN）→ 跳过。幂等：`{rKey}:OPEN` 决策行数据库唯一约束兜底，同周期同轮次绝不重复开仓。开仓成功后：写 TrendTradeMeta（featuresAtOpen 含 strategyId/modeTag/htf 等）、绑定决策行、调 `strategy.onOpen`、更新内存权益快照。

4.4 退出优先级（`manageExits`，从上到下短路）

优先级	规则	触发条件	exitReason
1	趋势确认结束	phase=ENDED 且 trendCycleId 与持仓一致 且 lastBarTime > meta.entryBarTime（证据必须晚于开仓 K 线）	TREND_ENDED
2	结束迹象报警	phase=END_PENDING 同周期 → **只记** firstEndSignAt，不平仓（要不要提前走交给策略）	—
3	**策略离场**	evaluateExit 返回 exit=true 且 reason 非空	策略给（SPIKE_TAKE/WEAKNESS_EARLY）
4	反向趋势确认	反方向 CONFIRMED_* 且 dataFresh	REVERSE_CONFIRMED

永不因「信号缺失（退池）/DATA_UNAVAILABLE」平仓——只由强平与人工/上述规则管理。强平是独立巡检（liquidationCheck，任何模式都跑），结果如实记 LIQUIDATED，不伪装成模型止损。

4.5 轮次体系速查

- 决策/待入场键：{cycleId}#R{round}；平仓键：{cycleId}#R{round}:CLOSE；跳过审计：{cycleId}:SKIP。
- 轮次号由策略维护（nextRound/onOpen），基础设施只透传；策略要保证「同周期递增、新周期归 1」。
- 同周期再入场惯例（short-swing 定下的三条，建议沿用）：距上次平仓 ≥ REENTRY_MIN_GAP_MS；出现晚于上次开仓依据的**新收盘 K 线**（有新证据，不是旧信号回声）；轮次上限自控（MAX_ROUNDS_PER_CYCLE）。

4.6 策略能做 / 不能做（红线清单的展开版）

能做：自维护价格样本与内部状态；自由定义入场相位门槛（score/ER/HTF/回撤/任何信号特征组合）；自由定义策略离场；决定同周期轮次上限与再入场条件；把状态落库（经 bindPersistence）。

不能做：接触真实下单（物理上也没有这个口子）；另算一套趋势方向（必须消费 UnifiedTrendSignal）；访问未来数据（evaluate 只能用当时可用快照；observe 样本也只能来自已发生的 mark）；阻塞主循环（observe/evaluate 内禁止 I/O、禁止长计算）；抛异常逃逸（所有策略入口都被设计为「绝不阻断交易循环」，但你也不该依赖这个兜底）；绕过基础设施自管资金/仓位/杠杆；修改 cfg 或全局配置；在 onOpen/onClose 里做交易决策。

---

5. 实战示例：手写一个新算法（pullback-v1 · 趋势回撤跟随）

5.1 算法思路（一段话）

与 short-swing 的「确认即追」不同，本示例做**趋势内回撤入场**：1m 趋势确认后不追价，等价格从短窗高点/低点顺势回撤 0.35%~1.2% 再入场（低吸/反抽）；回撤过深视为趋势被破坏、放弃。离场用「浮盈从峰值回撤」动态落袋——峰值直接复用基础设施采集的 `meta.mfe`，自己不算第二遍；叠加「走弱先走」。同周期限 2 轮。选币、方向、二次确认、资金、强平全部沿用基础设施。

> 命名与参数键约定：参数对象键统一带算法前缀 `PB_`——因为控制台参数键（PARAM_DEFS）是**全局命名空间**（§6.3），前缀防撞；即使暂不进控制台，这个习惯也为将来留了路。

5.2 完整代码（strategy/pullback.ts，可直接编译）

```
// ---------- 策略：趋势回撤跟随 v1（pullback-v1，接口文档 §5 教学示例） ----------
//
// 思路：异动选币交给候选池；1m 趋势确认（CONFIRMED_*）后不追价，等价格从短窗极值
// 顺趋势方向回撤一定幅度再入场（低吸/反抽）；回撤过深视为趋势被破坏、放弃。
// 离场：浮盈从峰值回撤即落袋（峰值复用基础设施 MFE 采集，不自己算第二遍），
// 叠加走弱先走（END_PENDING 同周期）。走强可按轮次再入场（≤2 轮）。
//
// 红线：纯模拟；阈值为教学初值、未经历史回放校准，不构成盈利能力声明；
// 强平/ENDED 周期匹配/反向确认/二次确认/冷却等基础设施规则不受本策略影响。

import type { UnifiedTrendSignal } from '../../trend-watch/signal';
import type { TrendExitReason } from '../config';
import type { TrendTradeMetaRow } from '../repo';
import type {
  StrategyEntryVerdict,
  StrategyExitVerdict,
  StrategyMarketCtx,
  TrendStrategy,
} from './types';

/** 趋势回撤跟随 v1 · 全部可调参数（唯一调参入口；教学初值）。
 * 键名带 PB_ 前缀：控制台参数键是全局命名空间，前缀防撞（文档 §6.3）。 */
export const PULLBACK = {
  /** 1m 趋势评分下限（挡杂音；趋势起点分数不高，别拿 60+ 当入场线） */
  PB_MIN_SCORE: 45,
  /** 1m 效率比下限（防磨损：震荡结构不参与） */
  PB_MIN_ER: 0.35,
  /** 回撤入场幅度（%）：短窗极值顺势回撤达到该值才入场（不追高） */
  PB_PULLBACK_PCT: 0.35,
  /** 最大容忍回撤（%）：超过视为趋势已被破坏，放弃入场 */
  PB_MAX_PULLBACK_PCT: 1.2,
  /** 回撤检测窗口（ms）：在该窗口内找极值作为回撤基准 */
  PB_LOOKBACK_MS: 20 * 60_000,
  /** 同一趋势周期最大开仓轮次 */
  PB_MAX_ROUNDS_PER_CYCLE: 2,
  /** 同周期再入场最小间隔（ms） */
  PB_REENTRY_GAP_MS: 60_000,
```

```
  /** 峰值回撤离场阈值（百分点，占保证金 %）：浮盈从峰值回撤 ≥ 该值 → 落袋 */
  PB_PEAK_RETRACE_EXIT: 1.5,
  /** 触发峰值回撤离场的最低浮盈门槛（占保证金 %；没赚到钱不谈回撤） */
  PB_PEAK_MIN_PROFIT: 2,
  /** 价格样本保留时长（ms；observe 采样粒度 ≈ SCAN_MS=10s） */
  PB_SAMPLE_TTL_MS: 30 * 60_000,
};

export type PullbackParams = Partial;

/** 合并参数（构造器覆盖 → 控制台/测试可调；未指定项用默认） */
function withDefaults(p?: PullbackParams): typeof PULLBACK {
  return { ...PULLBACK, ...(p ?? {}) };
}

/** 单 symbol 策略状态（持久化到 TrendStrategyState；重启恢复轮次语义） */
interface SymbolState {
  cycleId: string | null;
  round: number;
  lastEntryBar: number | null;
  lastExitAt: number | null;
}

/** 短窗价格样本 */
interface PriceSample { t: number; p: number }

export class PullbackStrategy implements TrendStrategy {
  readonly id = 'pullback-v1';
  readonly name = '趋势回撤跟随';
  readonly description =
    '异动选币不问方向；1m 趋势确认后不追高，等价格顺势回撤 0.35%~1.2% 再入场；' +
    '浮盈从峰值回撤 1.5 个百分点动态落袋；走弱先走、同周期可再入场（≤2 轮）';

  /** 可调参数（构造器覆盖；生产默认值） */
  readonly params: typeof PULLBACK;

  private states = new Map();
  private samples = new Map();
  /** 状态变化回调（AutoPaperTrader 注入：持久化到 TrendStrategyState） */
  private onStateChange: ((strategyId: string, symbol: string, json: string) => void) | null = null;

  constructor(params?: PullbackParams) {
    this.params = withDefaults(params);
  }

  bindPersistence(fn: (strategyId: string, symbol: string, json: string) => void): void {
    this.onStateChange = fn;
  }

  // ============ 行情采样（回撤检测的数据基础；全部为当时可用数据） ============

  observe(symbol: string, mark: number, now: number): void {
    if (!(mark > 0)) return;
    const arr = this.samples.get(symbol) ?? [];
    const last = arr[arr.length - 1];
    // 同一采样粒度内重复推送（<5s）覆盖末位，防 tick 重入导致样本堆积
    if (last && now - last.t < 5_000) {
```

```
    last.p = mark;
    last.t = now;
  } else {
    arr.push({ t: now, p: mark });
  }
  const cutoff = now - this.params.PB_SAMPLE_TTL_MS;
  while (arr.length > 0 && arr[0].t < cutoff) arr.shift();
  this.samples.set(symbol, arr);
}

/**
 * 当前价相对窗口极值的「顺势回撤幅度」（%）：
 * LONG = 距窗口最高点的回落深度；SHORT = 距窗口最低点的反弹深度。
 * 样本不足返回 0（不误判）；窗口 = 最新样本向前 PB_LOOKBACK_MS。
 */
private pullbackPct(symbol: string, side: 'LONG' | 'SHORT'): number {
  const arr = this.samples.get(symbol);
  if (!arr || arr.length < 2) return 0;
  const cur = arr[arr.length - 1]!.p;
  const target = arr[arr.length - 1]!.t - this.params.PB_LOOKBACK_MS;
  let extreme = cur;
  for (const s of arr) {
    if (s.t < target) continue;
    extreme = side === 'LONG' ? Math.max(extreme, s.p) : Math.min(extreme, s.p);
  }
  if (extreme <= 0 || cur <= 0) return 0;
  const pct = side === 'LONG'
    ? (extreme / cur - 1) * 100 // 高点比现价高多少 = 回落深度
    : (1 - extreme / cur) * 100; // 低点比现价低多少 = 反弹深度
  return pct;
}

// ============ 入场评估（确认相位内做回撤入场；方向由信号决定） ============

evaluateEntry(signal: UnifiedTrendSignal, _ctx: StrategyMarketCtx): StrategyEntryVerdict {
  // 相位守卫（基础设施已过滤，这里防御性再查一遍；方向唯一来自信号）
  if (signal.phase !== 'CONFIRMED_UP' && signal.phase !== 'CONFIRMED_DOWN') {
    return { want: false, side: null, modeTag: '-', reason: `趋势未确认（phase=${signal.phase}）` };
  }
  const side: 'LONG' | 'SHORT' = signal.phase === 'CONFIRMED_UP' ? 'LONG' : 'SHORT';
  const sym = signal.symbol;

  // 轮次约束（惯例三条：轮次上限 / 再入场间隔 / 新收盘 K 线证据）
  const st = this.states.get(sym);
  const sameCycle = st != null && st.cycleId === signal.trendCycleId;
  const nextRound = this.nextRound(sym, signal.trendCycleId ?? '');
  if (signal.trendCycleId && nextRound > this.params.PB_MAX_ROUNDS_PER_CYCLE) {
    return { want: false, side: null, modeTag: '-', reason: `本周期轮次已用尽（${st?.round}/${this.params.PB_MA
      X_ROUNDS_PER_CYCLE}）` };
  }
  if (sameCycle && st?.lastExitAt != null && Date.now() - st.lastExitAt < this.params.PB_REENTRY_GAP_MS) {
    return { want: false, side: null, modeTag: '-', reason: '同周期再入场间隔不足（刚离场）' };
  }
  if (sameCycle && st?.lastEntryBar != null && signal.lastBarTime != null && signal.lastBarTime <= st.lastEntry
      Bar) {
    return { want: false, side: null, modeTag: '-', reason: '同周期再入场等待新收盘 K 线（无新证据）' };
  }
```

```
  // 质量门槛：评分挡杂音、ER 挡磨损（均为 0-1/0-100 的信号自带特征）
  const score = signal.score;
  const er = signal.features.efficiencyRatio;
  if (score < this.params.PB_MIN_SCORE) {
    return { want: false, side: null, modeTag: 'PULLBACK', reason: `评分 ${score.toFixed(0)} < ${this.params.PB
      _MIN_SCORE}` };
  }
  if (er < this.params.PB_MIN_ER) {
    return { want: false, side: null, modeTag: 'PULLBACK', reason: `ER ${er.toFixed(2)} < ${this.params.PB_MIN_
      ER}` };
  }

  // 核心条件：顺势回撤进入 [PB_PULLBACK_PCT, PB_MAX_PULLBACK_PCT] 区间才入场
  const pb = this.pullbackPct(sym, side);
  if (pb < this.params.PB_PULLBACK_PCT) {
    return { want: false, side: null, modeTag: 'PULLBACK', reason: `回撤 ${pb.toFixed(2)}% 不足（<${this.params
      .PB_PULLBACK_PCT}%，不追高）` };
  }
  if (pb > this.params.PB_MAX_PULLBACK_PCT) {
    return { want: false, side: null, modeTag: 'PULLBACK', reason: `回撤 ${pb.toFixed(2)}% 过深（>${this.params
      .PB_MAX_PULLBACK_PCT}%，趋势疑似被破坏）` };
  }
  return {
    want: true, side, modeTag: 'PULLBACK',
    reason: `1m ${side === 'LONG' ? '看多' : '看空'}趋势内回撤 ${pb.toFixed(2)}% 入场 score=${score.toFixed(0)}
       er=${er.toFixed(2)}${nextRound > 1 ? ` 轮次#R${nextRound}` : ''}`,
  };
}

// ============ 离场评估（峰值回撤落袋 + 走弱先走；ENDED/反向/强平由基础设施处理） ============

evaluateExit(input: {
  meta: TrendTradeMetaRow;
  pos: { entryPrice: number; qty: number; margin: number; side: 'LONG' | 'SHORT'; openedAt: number };
  mark: number;
  signal: UnifiedTrendSignal | null;
  now: number;
}): StrategyExitVerdict {
  const { meta, pos, signal } = input;

  // 峰值回撤落袋：复用基础设施 MFE（按 tick 增量更新，无未来数据）——
  // 浮盈曾达 PB_PEAK_MIN_PROFIT% 且从峰值回撤 ≥ PB_PEAK_RETRACE_EXIT 个百分点 → 落袋
  if (meta.mfe != null && pos.margin > 0) {
    const sign = pos.side === 'LONG' ? 1 : -1;
    const pnlPct = ((input.mark - pos.entryPrice) * pos.qty * sign / pos.margin) * 100;
    const retrace = meta.mfe - pnlPct;
    if (meta.mfe >= this.params.PB_PEAK_MIN_PROFIT && retrace >= this.params.PB_PEAK_RETRACE_EXIT) {
      return {
        exit: true,
        reason: 'SPIKE_TAKE',
        detail: `峰值回撤离场：MFE ${meta.mfe.toFixed(1)}% → 当前 ${pnlPct.toFixed(1)}%（回撤 ${retrace.toFixe
      d(1)} 个百分点），落袋`,
      };
    }
  }
```

```
    // 走弱先走：本周期出现结束迹象即离场（异周期 END_PENDING 与本持仓无关，绝不误平）
    if (signal && signal.phase === 'END_PENDING' && signal.trendCycleId === meta.trendCycleId) {
      return {
        exit: true,
        reason: 'WEAKNESS_EARLY',
        detail: `短周期走弱先走（END_PENDING 证据 ${signal.endEvidenceBars} 根），走强后可按轮次再入场`,
      };
    }

    return { exit: false, reason: null, detail: '' };
  }

  // ============ 轮次状态（与 short-swing 同构） ============

  nextRound(symbol: string, cycleId: string): number {
    const st = this.states.get(symbol);
    if (!st || !cycleId) return 1;
    return st.cycleId === cycleId ? st.round + 1 : 1;
  }

  onOpen(symbol: string, cycleId: string, barTime: number | null, _now: number): void {
    const prev = this.states.get(symbol);
    const st: SymbolState =
      prev && prev.cycleId === cycleId
        ? { ...prev, round: prev.round + 1 }
        : { cycleId, round: 1, lastEntryBar: null, lastExitAt: null };
    st.lastEntryBar = barTime;
    this.states.set(symbol, st);
    this.flush(symbol);
  }

  onClose(symbol: string, cycleId: string, _reason: string, now: number): void {
    const prev = this.states.get(symbol);
    if (prev && prev.cycleId === cycleId) {
      this.states.set(symbol, { ...prev, lastExitAt: now });
      this.flush(symbol);
    }
    // 旧周期残响不动状态；持仓结束，样本清空（下轮持仓重建）
    this.samples.delete(symbol);
  }

  // ============ 持久化 ============

  exportState(): Record {
    const out: Record = {};
    for (const [sym, st] of this.states) out[sym] = st;
    return out;
  }

  importState(raw: unknown): void {
    this.states.clear();
    if (!raw || typeof raw !== 'object') return;
    for (const [sym, v] of Object.entries(raw as Record)) {
      if (!v || typeof v !== 'object') continue;
      const o = v as Partial;
      if (typeof o.cycleId !== 'string' || typeof o.round !== 'number') continue;
      this.states.set(sym, {
```

```
        cycleId: o.cycleId,
        round: o.round,
        lastEntryBar: typeof o.lastEntryBar === 'number' ? o.lastEntryBar : null,
        lastExitAt: typeof o.lastExitAt === 'number' ? o.lastExitAt : null,
      });
    }
  }

  private flush(symbol: string): void {
    if (!this.onStateChange) return;
    try {
      this.onStateChange(this.id, symbol, JSON.stringify(this.states.get(symbol) ?? {}));
    } catch {
      // 持久化失败不影响决策（状态仍在内存；重启后轮次可能回退一档——可接受）
    }
  }
}
```

5.3 逐段讲解（为什么这么写）

1. **参数对象 + `withDefaults`**：全部阈值集中在 `PULLBACK` 常量，`Partial` 作为覆盖类型——工厂传进来的控制台覆盖就是这个形状（§6.3）。`withDefaults` 一行合并，坏参数天然回落默认值。
2. `observe` **防堆积**：采样粒度 ≈10s（SCAN_MS），但同粒度可能重复推送——`<5s` 覆盖末位而不是追加，否则样本队列会被同一 tick 的重入撑爆。TTL 清理保证内存有界。这是 short-swing 踩过的坑，直接抄。
3. `pullbackPct` **只用样本**：窗口极值来自 observe 采集的**已发生**标记价，无未来数据；样本不足返回 0（不满足回撤条件，自然拒绝），绝不猜。
4. **入场四层**：相位（防御性）→ 轮次/再入场惯例（三条）→ 质量门槛（score/ER）→ 核心条件（回撤区间）。顺序有讲究：便宜的判断放前面，贵的（窗口扫描）放最后，evaluateEntry 每 tick 会被问很多次。
5. **离场复用** `meta.mfe`：基础设施已经逐 tick 维护了「持仓以来最大浮盈」，策略直接 `meta.mfe - 当前浮盈` 得到峰值回撤——不要自己再存一份峰值状态（两份数据必然漂移）。
6. **END_PENDING 必须做周期匹配**：`signal.trendCycleId === meta.trendCycleId`——异周期（比如本持仓是上一轮周期开的，当前信号已经是新周期）的走弱与本持仓无关，误匹配会把新趋势的好仓平掉。short-swing 的注释原话：「异周期 END_PENDING 与本持仓无关，绝不误平」。
7. **状态只记轮次四元组**：`(cycleId, round, lastEntryBar, lastExitAt)`——够用、可序列化、向后兼容好处理。importState 逐字段校验，脏数据丢弃；flush 吞异常。
8. **reason 全程人话 + 数字**：审计日志（§7.3）靠它排查，`评分 52 < 45` 这种格式一眼定位是哪道门槛拦的。

---

6. 接入上线四步（含 bump 纪律）

6.1 第一步 · 实现算法

1. 新建 src/lib/market/trend-trade/strategy/<你的算法>.ts（与 short-swing.ts 同目录平级）；
2. implements TrendStrategy，照 §5 示例或 short-swing.ts 的骨架写；漏一个成员 TypeScript 会立刻报错，编译期兜底；
3. 自查 §4.6 的能做/不能做清单与 §2.2 的逐方法红线；
4. 跑一次 bunx tsc --noEmit（或依赖编辑器报错）确认零错误——**此时不要动引擎**，未注册的文件是死代码，线上零影响。

6.2 第二步 · 注册（唯一入口）

```
// src/lib/market/trend-trade/strategy/registry.ts
import { PullbackStrategy, type PullbackParams } from './pullback'; // ← 新增 import

const STRATEGY_FACTORIES: Record) => TrendStrategy> = {
  'short-swing-v1': (p) => new ShortSwingStrategy(p as ShortSwingParams | undefined),
  'pullback-v1': (p) => new PullbackStrategy(p as PullbackParams | undefined), // ← 新增一行
};
```

规则与语义：

- **id 命名**：{算法名}-v{序号} 全小写连字符（如 pullback-v1）。id 会被持久化到 TrendAutoConfig.strategyId 并作为策略状态表的命名空间——**上线后不可改名**；日后行为变更开 xxx-v2 新 id 而不是改 v1。
- **UI 自动出现**：策略下拉与切换确认弹窗读 listStrategies()（id/name/description），注册即见，无需改前端。
- **未知 id 兜底**：createStrategy 对未注册 id 回落 DEFAULT_STRATEGY_ID（short-swing-v1），绝不抛错阻断交易循环——所以注册表改坏时的爆炸半径 = 退回默认算法，可控。
- 同一时刻只有一个策略在跑（TrendAutoConfig.strategyId 全局一行），没有组合/加权等花活；要并行跑两个算法，以后再加多账户。

6.3 第三步（可选）· 参数接进控制台

不接也能跑：工厂收到 undefined 参数 → withDefaults 全用代码默认；想调就改代码 + bump（§6.4）。要让用户在页面上热调参数，才做本步：

```
// src/lib/market/trend-trade/console-params.ts · PARAM_DEFS 数组追加
{
  key: 'PB_MIN_SCORE', scope: 'strategy', group: '回撤跟随 · 入场',
  label: '评分下限', type: 'number', default: PULLBACK.PB_MIN_SCORE,
  min: 0, max: 100, step: 1,
  desc: '1m 趋势评分达到该值才允许回撤入场。趋势起点分数不高，本门槛只挡杂音。',
  effect: '调低 → 更早入场、单量变多；调高 → 只做证据更强的回撤',
},
```

三个必须知道的机制：

1. **键是全局命名空间**：PARAM_DEFS 的 key 跨策略唯一（校验索引 DEF_BY_KEY 是单表），所以新策略参数键必须带算法前缀（PB_），否则与 short-swing 的 ENTRY_MIN_SCORE 等撞名——撞了不会报错，而是**静默错用对方的定义**（范围/文案全错），这比报错危险得多。
2. **未注册的键在装配时被静默丢弃**：applyStrategy 用 sanitizeRuntimeParamsLenient 清洗 DB 覆盖项——不在 PARAM_DEFS 里的键直接丢弃（回退代码默认）。也就是说：**参数不进 PARAM_DEFS = 控制台写了也不生效**。
3. **热生效链路**：控制台「保存」→ control API 严格校验（白名单/类型/范围，非法直接 400 报错给用户）→ strategyParamsJson 按 strategyId 分桶落库 → applyStrategy 重装配实例（createStrategy(id, 覆盖) + importState 从 DB 恢复轮次）→ 下一 tick 生效。infra 域参数（waitBarConfirm/cooldownMs/marginRatio）走配置列，每 tick 重读，同样下 tick 生效；marginRatio 实际生效值 = min(设定值, 95% ÷ 持仓数)。

6.4 第四步 · bump 引擎单例（改行为必 bump，铁律）

为什么：enhanced 引擎（含 AutoPaperTrader 与策略实例）以 globalThis.__binanceRadarEnhanced_v15 驻留进程。dev server 热重载只替换模块代码，不替换 globalThis 上的旧实例——不 bump，你的新策略代码永远跑在旧实例的旧闭包里，看起来「改了没生效」。bump = 换 key = 强制用新代码重建实例；旧实例 stop() 被链式调用（停掉全部定时器，含 autoTrader 主循环），新实例从 DB reconcile 恢复持仓关联与轮次状态。**全程无需重启 server 3000**。

```
// src/lib/market/enhanced/engine.ts —— 共四处
const g = globalThis as typeof globalThis & {
  // ...v2~v15 声明原样保留...
  __binanceRadarEnhanced_v16?: EnhancedEngine; // ① 类型声明加一行
};
// ② pickLegacyEngine 回收链补一段 v15 → v16（旧实例停表 + 状态/K线仓库交接）
// ③ getEnhancedEngine / resetEnhancedEngine 的读写 key 切到 v16
// ④ 版本注释头加一条：
// v16（Task 6x）：注册 pullback-v1（趋势回撤跟随）……改行为必 bump key；
// 旧实例 stop() 会同步停掉 autoTrader 主循环，新实例从 DB 恢复持仓关联（reconcile）。
```

记忆口径：**凡是 strategy/、registry、auto-trader、console-params、config 里的任何改动（哪怕只是加一个新策略、调一个默认阈值）→ bump v15→v16**。没 bump 的改动 = 没发生。

6.5 验证与回滚（不重启 server）

验证清单（顺序执行，全部被动观察，不 reset 引擎）：

1. 等热重载完成（改完保存后 ~3-10s）；
2. curl -s localhost:3000/api/trend-trading | jq '.config.strategies' → 新 id 出现在列表，strategyId 仍为旧值（未切换）；
3. curl -s localhost:3000/api/trend-trading | jq '.config' → 交易循环存活：autoTrader.on = true、lastTick 在 10s 内刷新；

4. UI 下拉出现新算法 → 点切换 → 确认弹窗（显示 name/description + 清场提示）→ 返回 note「算法已切换为 …；已自动平仓 N 笔」；
5. 日志观察：`[trend-auto]` 前缀的决策日志开始出现新算法的 modeTag/reason；策略跳过记录里能看到「策略不入场：{你的 reason}」字样——说明 evaluateEntry 真的在跑；
6. 观察一个完整周期（开仓 → 离场 → 轮次），确认 `TrendTradeMeta.featuresAtOpen.strategyId` = 新 id。

回滚：UI 切回旧算法（同样自动清场）；若要彻底下线代码，还原 registry 一行 + bump 一次（v16→v17，对，回滚也是行为变更也要 bump）。

---

7. 运行期管理：切换、调参与审计

7.1 切换算法（UI / control API）

- 入口：趋势交易页顶部算法下拉 → 选择 → 确认弹窗（列出新算法的 name/description 与「先清场」提示）→ `POST /api/trend-trading/control {action: 'setStrategy', value: ''}`。
- 服务端流程：`isKnownStrategy` 校验（未知 id 400）→ 同值 no-op → **先 `closeAllManual` 全部平仓清场**（不同算法离场规则不同，持仓不交给新算法接管）→ `setStrategyId`（持久化 strategyId + 热装配：参数覆盖合并 → bindPersistence → importState 恢复轮次）。
- 清场副作用：所有待入场记录随清场失效；切换是「换脑子」不是「接力」，这一点在 UI 文案里已明示。

7.2 参数控制台

- 位置：趋势交易页参数控制台卡片，按 `PARAM_DEFS` 的 group 分组渲染，含 label/当前值/默认值/单位/悬浮说明（desc + effect）。
- 语义：保存即热生效（§6.3 机制 3）；「恢复默认」清掉当前策略的覆盖分桶 + infra 三项回默认（轮次状态保留）；当前值 ≠ 代码默认的项带「已自定义」徽标。
- 边界：参数改动不触发清场，只影响后续决策；二次确认（waitBarConfirm）开关对「策略想开仓之后」的流程生效，1m 趋势确认本身固定需要 3 根已收盘 K 线，不在可调范围。

7.3 审计三查（排查「为什么没开/为什么开了/为什么平了」）

1. **决策日志**：服务端控制台 grep `[trend-auto]`——开仓/平仓/待入场取消/策略离场/参数热应用全部结构化落一行（含 strategyId、modeTag、round、score、er、htf、mfe/mae），一次误操作可完整还原。
2. **跳过记录**：`GET /api/trend-trading` 返回 `recentSkips`（最近 50 条）——「策略不入场：{reason}」就是你的 evaluateEntry 拒绝文案；「已达最大持仓数」「同币种冷却中」「数据不可用」等是基础设施拦的。
3. **DB 决策行**：`TrendAutoDecision`（每次开/平/跳的幂等与审计行，detail JSON 含 strategyId/modeTag/净值）、`TrendTradeMeta`（每笔持仓的开仓快照与 MFE/MAE）、

`TrendStrategyState`（策略 per-symbol 状态 JSON）。`TrendAutoDecision.cycleId` 里的 `#R{n}` 后缀就是轮次。

7.4 FAQ

- **为什么一直没开仓？** 先查 recentSkips：90% 是「策略不入场：{reason}」——那是你自己的门槛在拦，数字就写在 reason 里；其余是容量/冷却/数据三类基础设施拦截。
- **为什么我的 evaluateEntry 没被调用？** 它只会在「确认相位 + dataFresh + 无持仓 + 无待入场 + 容量冷却通过」时被问（§4.1 守卫链）。候选池里根本没这个币、或趋势还没确认，策略不会被问——这是设计，不是 bug。
- **切换算法后旧持仓怎么办？** 已全部平仓（清场），新算法从空仓开始；待入场记录同步作废。
- **参数改了没生效？** 三查：键是否已在 PARAM_DEFS 注册（§6.3 机制 2，未注册 = 静默丢弃）；是否属于 infra 域（下 tick 生效，不会立刻出现在已持有的仓位上）；控制台保存是否返回了 400（严格校验会拒绝非法值并显示原因）。
- **强平和策略离场谁先？** 强平是独立巡检（每 tick 第一步），永远最优先；策略离场排在 ENDED 之后、反向确认之前（§4.4）。
- **重启/崩溃后会怎样？** 全状态在 DB：持仓/待入场/配置/策略轮次状态都恢复；唯一可能丢失的是 observe 内存样本（重启后短窗策略前 ~10 分钟检测偏保守，可接受）。

---

8. 红线清单与上线自测

8.1 红线（逐条自查）

1. **纯模拟**：绝不接真实下单；一切只作用于独立账户 trend-auto-v1；手动盘 default 完全不可触达。
2. **无未来数据**：决策只用当时可用快照（信号/K 线已收盘部分/observe 已采样 mark/meta 增量统计）；不预读、不回填重构「当时本可知道」的数据。
3. **绝不阻塞**：observe/evaluateEntry/evaluateExit 内禁止 I/O、禁止长计算；主循环 10s 一拍，卡住它 = 全账户风险管理停摆。
4. **绝不抛错逃逸**：所有策略入口要求「永不阻断交易循环」；importState/flush 必须自吞脏数据与异常。
5. **状态可恢复**：轮次语义必须经 exportState/importState 持久化自洽；恢复失败按首次处理（保守，不丢单）。
6. **周期匹配**：任何与 trendCycleId 相关的判断（尤其 END_PENDING 离场）必须匹配持仓周期，绝不「只凭 symbol」行动。
7. **改行为必 bump**：strategy/registry/auto-trader/console-params/config 任何改动 → engine.ts 单例 key bump + 版本注释；server 3000 永不重启。
8. **阈值声明纪律**：新算法阈值必须标注「初始工程参数、未经历史回放校准」，不构成盈利能力声明——与 short-swing 同一口径。

8.2 上线前自测清单

- [ ] `tsc --noEmit` 零错误；新文件被 registry import（否则是死代码）
- [ ] id 符合 `{算法名}-v{n}` 且不与现存 id 冲突；name/description 是人话
- [ ] evaluateEntry 对 phase 防御性校验（虽然基础设施已过滤）
- [ ] want=true 的分支 reason 里带数字证据；want=false 全分支有中文拒绝理由
- [ ] END_PENDING 离场做了 `signal.trendCycleId === meta.trendCycleId` 匹配
- [ ] 轮次三惯例（上限/间隔/新 K 线证据）已实现；onOpen/onClose 与 nextRound 自洽（同周期递增、新周期归 1）
- [ ] observe 样本有界（TTL/上限清理 + 重入覆盖）；无 I/O
- [ ] exportState/importState 往返一致（手测一次 JSON round-trip）
- [ ] 参数键带算法前缀且已在 PARAM_DEFS 注册（若要控制台可调）
- [ ] engine.ts bump 四处完成（声明/回收链/读写 key/注释）；未重启 server
- [ ] 验证清单 §6.5 六步通过；观察至少一个完整开平周期后收工

8.3 版本记录

版本	日期	内容	对应代码状态
v1	2026-10-17	初版：接口/契约/守卫链/示例/接入四步	enhanced 引擎单例 v15 · paper v9 · paperAuto v2 · 默认策略 short-swing-v1